Manual 2: Getting Started with CESNET Kubernetes (e-INFRA CZ / CERIT-SC)

Pollination Ecology Group (Handrkov) — Rancher, kubectl, and Security Rules

2026-11-12

Table of contents

1. Getting Access to e-INFRA CZ & Rancher . . . 2
   - Step 1: Ensure You Have an Active e-INFRA CZ / MetaCentrum Account . . . 2
   - Step 2: Log into the Rancher Web UI . . . 2
2. Setting Up `kubectl` on Your Computer . . . 2
   - Step 1: Install `kubectl` . . . 2
   - Step 2: Download `kubeconfig.yaml` from Rancher . . . 3
   - Step 3: Set Your Default Namespace to `stenc-ns` and Test Connection . . . 3
3. Mandatory CERIT-SC Security Rules (Read Before Creating Pods!) . . . 3
   - Rule 1: Must Run as a Non-Root User (`UID 1000`) . . . 4
   - Rule 2: Must Drop Linux Capabilities & Privilege Escalation . . . 4
   - Rule 3: Must Mount a Writable `/app` or `/tmp` Directory (`emptyDir`) . . . 4
   - Rule 4: Always Specify Resource `requests` and `limits` . . . 4
4. Namespace Resource Quotas & CERIT-SC Limits . . . 5
   - Important Quota Tips for Colleagues: . . . 5
5. Essential `kubectl` Cheat Sheet . . . 6
6. Common Pod Statuses Explained . . . 6

Our group uses the **CERIT-SC / e-INFRA CZ Kubernetes platform** (`rancher.cloud.e-infra.cz`) to run compute-intensive R and Python scripts, train deep-learning models on NVIDIA GPUs, and host interactive web applications.

This manual walks you through:

1. Logging into the Rancher web dashboard and getting access to our namespace (`stenc-ns`).
2. Downloading your `kubeconfig.yaml` credentials file and installing the `kubectl` command-line tool.
3. Understanding the **mandatory CERIT-SC security policies** (why pods fail if you don't set `runAsNonRoot: true` and how to fix it).
4. The essential `kubectl` command cheat sheet.

---

1. Getting Access to e-INFRA CZ & Rancher

Step 1: Ensure You Have an Active e-INFRA CZ / MetaCentrum Account

1. All academic staff and students at Czech universities (Charles University, CAS, etc.) can register for free at e-infra.cz.
2. To access the Kubernetes cluster (`kuba-cluster`) and our shared project namespace (`stenc-ns`), ask Jakub to add your e-INFRA CZ / Perun username to the project namespace if you don't see it yet.

Step 2: Log into the Rancher Web UI

1. Open your browser and go to: **https://rancher.cloud.e-infra.cz**
2. Click **Log in with Shibboleth / e-INFRA CZ** and sign in using your institutional login (e.g., Charles University CAS / eduID.cz).
3. In the Rancher home dashboard, click on the cluster **`kuba-cluster`**.
4. In the top namespace filter bar, select **`stenc-ns`** so you only view our group's Pods, Jobs, Deployments, ConfigMaps, and Secrets.

---

2. Setting Up `kubectl` on Your Computer

While you can view logs and pods in the Rancher web browser, submitting scripts and streaming logs is much faster from your terminal using `kubectl`.

Step 1: Install `kubectl`

- **Linux (Ubuntu / Debian):**

```
curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/
chmod +x kubectl
sudo mv kubectl /usr/local/bin/
```

- **macOS (Homebrew):**

```
brew install kubectl
```

- **Windows (PowerShell / Winget):**

```
winget install -e --id Kubernetes.kubectl
```

Step 2: Download `kubeconfig.yaml` from Rancher

1. In the Rancher web UI (https://rancher.cloud.e-infra.cz), open **`kuba-cluster`**.
2. In the **top-right corner** of the screen, click the **Document icon ("Download KubeConfig")**.
3. Save the downloaded file as `~/.kube/config` (Linux/macOS) or `C:\Users\<YourUser>\.kube\config` (Windows):

```
mkdir -p ~/.kube
mv ~/Downloads/kuba-cluster.yaml ~/.kube/config
chmod 600 ~/.kube/config
```

*(Alternatively, if you keep **`kubeconfig.yaml`** inside a project folder, pass **`--kubeconfig kubeconfig.yaml`** or run **`export KUBECONFIG=/path/to/kubeconfig.yaml`**.)*

Step 3: Set Your Default Namespace to `stenc-ns` and Test Connection

Run this one-liner so you don't have to type `-n stenc-ns` on every single command:

```
kubectl config set-context --current --namespace=stenc-ns
```

Now test your connection:

```
kubectl get pods -n stenc-ns
```

If you see a list of running/completed pods (or `No resources found in stenc-ns namespace`), your connection is working!

> **Tip**
>
> **Troubleshooting `error: You must be logged in to the server (Unauthorized)`:** For security, Rancher API tokens inside `kubeconfig.yaml` expire periodically (every 30–90 days). Whenever you see `Unauthorized`, simply log into https://rancher.cloud.e-infra.cz, click **Download KubeConfig** again, and replace `~/.kube/config`.

---

3. Mandatory CERIT-SC Security Rules (Read Before Creating Pods!)

CERIT-SC enforces strict **Pod Security Standards (`restricted`)** to protect the shared university infrastructure. **If you copy a generic Kubernetes YAML example from the internet, the cluster will reject it immediately** unless it includes the following four settings:

Rule 1: Must Run as a Non-Root User (`UID 1000`)

Containers are **forbidden from running as `root` (`UID 0`)**. Every Pod spec must include this `securityContext` at the **Pod level** (`spec.template.spec.securityContext`):

```yaml
      securityContext:
        runAsNonRoot: true
        runAsUser: 1000
        runAsGroup: 1000
        seccompProfile:
          type: RuntimeDefault
```

Rule 2: Must Drop Linux Capabilities & Privilege Escalation

Every container inside `containers:` must include this `securityContext` at the **Container level**:

```yaml
        securityContext:
          allowPrivilegeEscalation: false
          capabilities:
            drop:
            - ALL
```

Rule 3: Must Mount a Writable `/app` or `/tmp` Directory (`emptyDir`)

Because your container runs as user `1000` (not `root`), it **cannot write files to `/` or `/root`** and cannot install packages globally (`pip install` or `install.packages()` into `/usr/local/lib` will fail with `Permission denied`). To solve this cleanly:

- Mount an `emptyDir: {}` volume at `/app`.
- Set `HOME=/app` (and `R_LIBS_USER=/app/R_libs` for R, or `pip install --user` with `PATH=/app/.local/bin:$PATH` for Python).
- All our templates in `templates/k8s/` already configure this automatically!

Rule 4: Always Specify Resource `requests` and `limits`

Every container should state how much CPU, RAM (`memory`), and optionally GPU (`nvidia.com/gpu`) it needs:

```yaml
        resources:
          requests:
            cpu: "4"
            memory: "8Gi"
          limits:
            cpu: "8"
            memory: "16Gi"
```

- **Important:** If your R or Python script uses more RAM than `limits.memory`, Kubernetes will terminate the container immediately with status `OOMKilled` **(Exit Code 137)**. If that happens, increase `memory` in your YAML (e.g., from `16Gi` to `32Gi`).

---

4. Namespace Resource Quotas & CERIT-SC Limits

Every namespace on CERIT-SC Kubernetes has a total resource quota (`ResourceQuota`) shared across all **currently running** Pods in that namespace (`Completed`/`Succeeded` jobs do not consume quota):

Resource	Personal Namespace Quota (`stenc-ns` / `<user>-ns`)	Group Project Quota (on request via `k8s@cerit-sc.cz`)	Description
`requests.cpu`	**20 CPU cores**	12–64+ CPU cores	Guaranteed CPU cores reserved across all running pods
`limits.cpu`	**32 CPU cores**	**64+ CPU cores**	Maximum burst CPU ceiling across all running pods
`requests.memory`	**40 GiB RAM**	40–256+ GiB RAM	Guaranteed RAM reserved across all running pods
`limits.memory`	**64 GiB RAM**	**256+ GiB RAM**	Maximum RAM ceiling across all running pods *(includes `/dev/shm` memory volumes)*
`nvidia.com/gpu`	**Uncapped in quota** *(limited by CPU/RAM quota)*	**Uncapped in quota**	Multiple GPUs can run concurrently as long as pod CPU/RAM fits in the quota
Default per container	`1 CPU / 512 MiB`	`1 CPU / 512 MiB`	Applied automatically by `LimitRange` if `resources:` is omitted

Important Quota Tips for Colleagues:

1. **Check Available Quota Before Submitting Large Jobs:**

   ```
   kubectl describe resourcequota -n stenc-ns
   ```

   If `limits.cpu` or `limits.memory` is near 100%, a new job will wait until running jobs finish.

2. **Use Your Own Personal Namespace (`<username>-ns`) for Extra Capacity:** When you log into https://rancher.cloud.e-infra.cz, CERIT-SC automatically creates your own personal namespace (`<surname>-ns`) with its own independent **20/32 CPU and 40/64 GiB RAM** quota! You can copy the `s3-credentials` Secret into your personal namespace and run jobs using your own quota while still reading and writing data to our shared `peg-bucket` S3 storage.

3. **Idle GPU Pruning Policy:** Interactive pods/notebooks requesting a GPU with `< 0.005` (0.5%) GPU utilization for 2 consecutive days are automatically terminated by CERIT-SC so GPUs are not locked while idle. Always use `kind: Job` for GPU training so the GPU is released as soon as your script finishes.

---

5. Essential `kubectl` Cheat Sheet

All commands below assume you are working in namespace `stenc-ns`:

What You Want to Do	Command
List all pods and their status	`kubectl get pods -n stenc-ns`
List all batch jobs	`kubectl get jobs -n stenc-ns`
Submit / start a Job or Deployment	`kubectl apply -f my-job.yaml`
Stream live output logs from a Job	`kubectl logs -f job/<job-name> -n stenc-ns`
Stream live logs from a specific Pod	`kubectl logs -f <pod-name> -n stenc-ns`
See why a Pod is stuck in `Pending` or crashed	`kubectl describe pod <pod-name> -n stenc-ns`
Open an interactive bash terminal inside a running Pod	`kubectl exec -it <pod-name> -n stenc-ns -- bash`
Delete a finished or failed Job	`kubectl delete job <job-name> -n stenc-ns`
List ConfigMaps (uploaded scripts)	`kubectl get configmaps -n stenc-ns`
List Secrets (S3 & app credentials)	`kubectl get secrets -n stenc-ns`
Check CPU & RAM usage of running Pods	`kubectl top pods -n stenc-ns`

---

6. Common Pod Statuses Explained

When you run `kubectl get pods -n stenc-ns`, look at the `STATUS` column:

- `Pending` / `ContainerCreating`: Kubernetes is finding a node with enough free CPU/RAM/GPU and downloading the Docker image. If it stays `Pending` for >5 minutes, run `kubectl describe pod <pod-name> -n stenc-ns` and look at the bottom `Events:` section (e.g., all GPUs on the cluster might currently be busy).
- `Running`: Your script is actively executing! Run `kubectl logs -f <pod-name> -n stenc-ns` to watch its progress.
- `Completed`: Your Job finished with Exit Code `0` (success).
- `Error` / `CrashLoopBackOff`: Your script threw an exception or syntax error. Run `kubectl logs <pod-name> -n stenc-ns` to read the R/Python traceback.
- `OOMKilled`: Your script exceeded the `memory` limit defined in your YAML. Increase `resources.limits.memory` and re-submit.